



Google Summer of Code



GSoC 2026 Project Proposal for
**Neural Operators for Fast
Simulation of Strong
Gravitational Lensing**
(Duration: 175 hours)

SNEHANSHU MUKHERJEE

1 Contributor Information.....	3
2 Abstract.....	3
3 Technical Details.....	3
3.1 Neural Operators and Gravitational Lensing Simulation.....	3
3.2 Why Fourier Neural Operator?.....	5
3.2 Evaluation Task.....	5
3.1.1 Common Task: Image Classification of Lenses.....	5
3.1.2 Specific Task 4: Training Neural Operator for Image Classification.....	6
4 Project Deliverables.....	8
4.1 Objective and Implementation details.....	9
4.1.1 Literature Review.....	9
4.1.2 Dataset Preparation.....	9
4.1.3 Core Training Phase.....	9
4.1.4 Advanced FNO Experimentation.....	9
4.1.5 Evaluation Metrics.....	10
4.1.6 Physics & Generalization Benchmarking.....	10
4.1.7 Documentation.....	10
4.1.8 Extension.....	10
4.2 Detailed Timeline.....	10
5 Other Information.....	13
5.1 Why ML4Sci?.....	13
5.2 Relevant Experience.....	13
5.3 Commitments.....	14
6 References.....	15

1 Contributor Information

Name: Snehanshu Mukherjee

Email: snehanshumukh@gmail.com, 21je0928@iitism.ac.in

Time Zone: Indian Standard Time (+5:30 GMT)

GitHub: [pilot-j](#)

LinkedIn: [Snehanshu](#)

Resume: [Snehanshu_CV](#)

University: Indian Institute of Technology (ISM), Dhanbad

Major: Electrical Engineering

Minor: Data Science

Graduated: May 2025

Degree: Bachelor of Technology

2 Abstract

Simulating strong gravitational lensing is essential for studying dark matter, but traditional numerical ray-tracing methods are slow and computationally heavy. This project proposes building a fast, deep learning based simulator using Fourier Neural Operators (FNOs).

In lensing, calculating how light bends is fundamentally a spatial convolution. FNOs are a perfect fit for this problem because they evaluate convolutions in the frequency domain. This means the network's internal math naturally aligns with the actual physics of gravitational deflection.

My goal is to train an FNO that directly maps mass distributions and background light sources to the final observed images. Over the duration of the program, I plan integrate physics-based constraints into the model and compare its speed and accuracy against baselines like DeepONets, standard CNNs, and previous diffusion models. The final deliverable will be an open-source package for DeepLense that creates accurate lensing simulations and is faster than current numerical methods.

3 Technical Details

3.1 Neural Operators and Gravitational Lensing Simulation

Gravitational lensing occurs when massive celestial objects bend light from background sources, creating distorted or multiple images. This phenomenon offers a powerful observational tool to infer the composition and structure of dark matter.

Traditional gravitational lensing simulations rely on ray tracing to model how light bends around massive objects. However, these methods require detailed mass distribution modeling and are computationally intensive and time-consuming. At its core, given a certain input mass distribution and source, simulating a lensed image requires solving a system of partial differential equation. A new paradigm that has evolved in solving PDEs is neural operator - where a deep learning based network is trained to learn the operator between two function spaces.

DeepONets [1] build upon the Universal Approximation Theorem [2] for operators to learn nonlinear mappings between function spaces. The model comprises a "branch net" that encodes the input function observed at a fixed, discrete set of sensor locations, and a "trunk net" that embeds the continuous spatial or temporal coordinates of the target domain. The final operator evaluation is constructed by taking the dot product of these latent representations:

$$G(u)(y) \approx \sum_{k=1}^p b_k(u(x_1), \dots, u(x_m)) t_k(y) \dots \dots (1)$$

Neural Operators [3] learn a discretization-invariant mapping between infinite-dimensional spaces, explicitly removing DeepONet's fundamental assumption of fixed, discrete sensor locations. They achieve this mesh-free property by generalizing standard network layers to continuous domains, replacing discrete weight matrices with integral operators. The hidden representation $v_t(x)$ is iteratively updated via the formulation

$$v_{\{t+1\}}(x) = \sigma(W v_t(x) + \int_D K(x, y) v_t(y) dy) \dots \dots (2)$$

where K is a learnable kernel parameterized by a neural network, W is a weights matrix (linear transformation) and σ is activation function (usually $GeLU()$ [4]). This continuous formulation ensures the learned operator strictly respects the underlying functional spaces, allowing it to be trained on one grid resolution and evaluated at any arbitrary or finer resolution without retraining.

Fourier Neural Operators [5] (FNOs) are special category of operators that instantiate the continuous integral operator by restricting the kernel to be *shift-invariant*. FNOs reformulate the spatial integration as a continuous convolution. Using the Convolution Theorem [6], they bypass the dense spatial integration and evaluate the integral directly in the frequency domain (Using Fast Fourier Transform [7]). The iterative update is explicitly computed as:

$$v_{\{t+1\}}(x) = \sigma(W v_t(x) + \mathcal{F}^{-1}(R \cdot \mathcal{F}(v_t))(x)) \dots \dots (3)$$

where $\mathcal{F}$ denotes the Fourier transform and R is a learnable complex weight tensor that acts on the truncated frequency modes. This spectral formulation reduces computational complexity to neural operator approach from $O(N^2)$ to $O(N * \log(N))$

In the context of strong gravitational lensing, a simulator maps a foreground mass convergence field $\kappa(\theta)$ and a background source light profile $I_{source}(\beta)$ to an observed lensed image $I_{obs}(\theta)$. This project defines two primary simulation targets:

- Mapping $\kappa(\theta) \rightarrow I_{lensed}(\theta)$ against a fixed background source. This serves as a diagnostic objective allowing us to isolate and evaluate the learned operator's behavior.
- Composite mapping $(\kappa(\theta), I_{source}(\theta)) \rightarrow I_{obs}(\theta)$ with variable source light profiles.

The underlying physical system is governed by a strict set of spatial equations. Given a mass convergence field κ the lensing potential ψ is derived by solving the 2D Poisson equation:

$$\nabla^2 \psi(\theta) = 2\kappa(\theta) \dots\dots\dots (4)$$

The deflection angle α experienced by the light rays is the gradient of this potential:

$$\alpha = \nabla\psi \dots\dots\dots (5)$$

The lens equation maps the observed image plane coordinates θ back to the unlensed source plane coordinates β :

$$\beta = \theta - \alpha(\theta) \dots\dots\dots (6)$$

Traditional forward simulators [8][9] rely on discrete numerical solvers to compute this sequence step-by-step.

The Neural Operator approach fundamentally collapses this multi-step numerical pipeline. Instead of sequentially computing the discrete potential, deflection, and coordinate mapping, the model learns the continuous solution operator $\mathcal{G}$ to the governing PDE system.

$$\mathcal{G}[\kappa, I_{source}](\theta) = I_{source}(\theta - \nabla\psi_K(\theta)) \dots\dots\dots (7)$$

3.2 Why Fourier Neural Operator?

In this project, I plan to extensively focus on FNO as the primary simulation pipeline. This is a conscious choice. Analytically, solving for the deflection angle $\alpha(\theta)$ from the convergence field is a pure spatial convolution with a known kernel: $\alpha(\theta) = \frac{\theta}{\pi|\theta|^2} * \kappa(\theta)$.

Convolution in spatial domain = multiplication in Fourier domain. FNO learns directly in Fourier space, making it architecturally aligned with the physics.

3.2 Evaluation Task

The key task was training an image classification model on the provided dataset. All related notebooks and code can be found here - https://github.com/pilot-j/neural_operator_Gsoc_2026

Dataset - The dataset consists of gravitational lensing images across three classes: No Substructure, Sphere (subhalo) Substructure, and Vortex Substructure. It is organized into train and validation splits. The train set is further divided into 90% for training and 10% for validation. The final evaluation metrics are reported on the held out validation set.

Evaluation Metric - ROC curve, AUC score

3.1.1 Common Task: Image Classification of Lenses

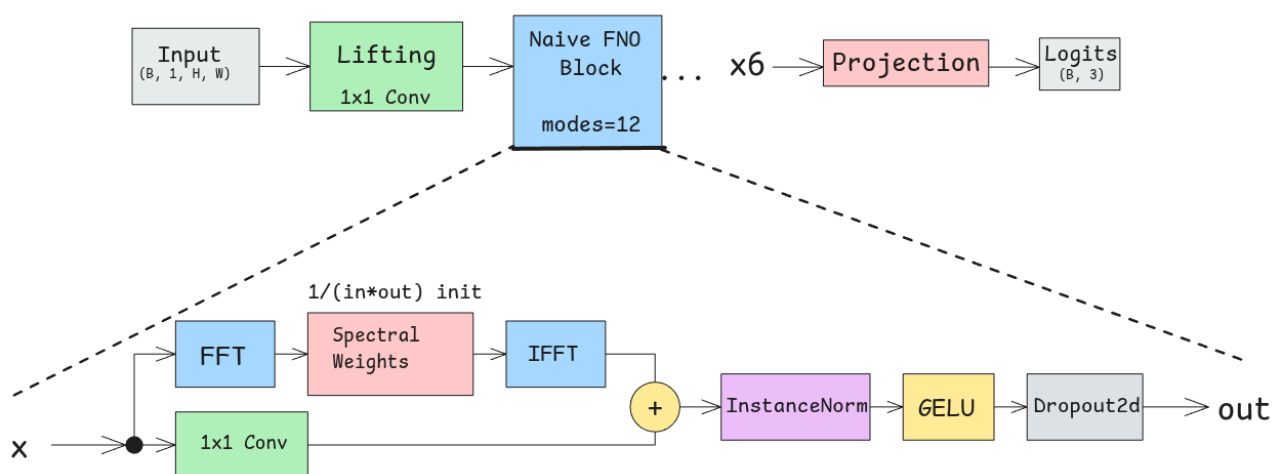
- A ResNet-18 backbone was used for classification, augmented with Squeeze-and-Excitation [10] blocks to improve performance. The final model has 13.6M parameters and achieves a micro AUC of **0.9908** [no: 0.9905, sphere:0.9824, vort:0.9936]
- Training was performed without data augmentation, aside from resizing images for input compatibility. I used crossentropy() loss and a learning rate of 1e-4.

3.1.2 Specific Task 4: Training Neural Operator for Image Classification

I chose to implement and test only fourier neural operator for this task (reason mentioned in 3.2). I systematically tested 3 variations of architecture as below -

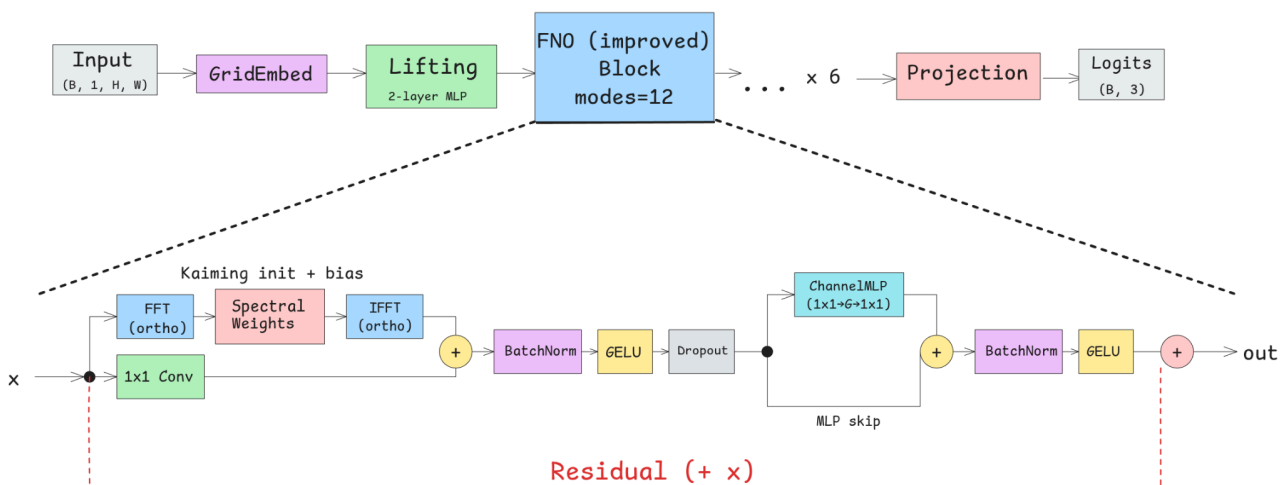
1. **Naive FNO** - Minimal FNO implementation: input passes through stacked spectral blocks where each block performs FFT $\rightarrow$ learned spectral weights $\rightarrow$ IFFT, combined with a parallel 1x1 convolution. Uniform weight initialization, and InstanceNorm. This sets our baseline score for FNO

Naive FNO Architecture



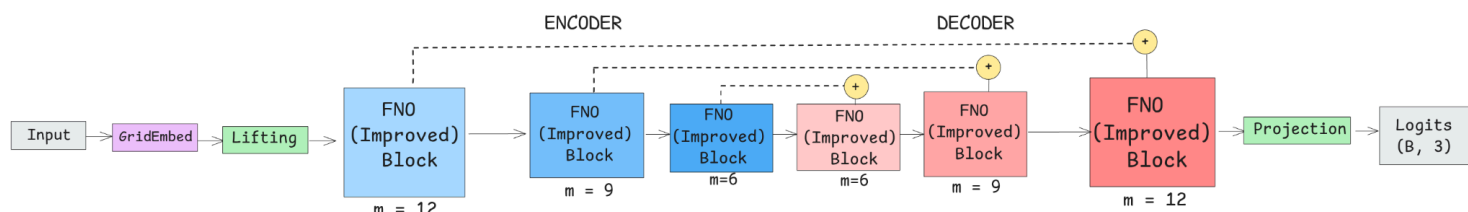
2. **FNO (Improved)** - Building over naive version I added positional encoding and skip connections mimicking ResNet. This implementation is inspired by current version as in neural operator library. The lifting layer introduced MLP based channel level mixing. I moved from uniform to Kaiming initialisation [11] of weight (faster). I tested with two frequency modes = 14, 12. *Fno_ce_light* has modes = 12

FNO_CE light



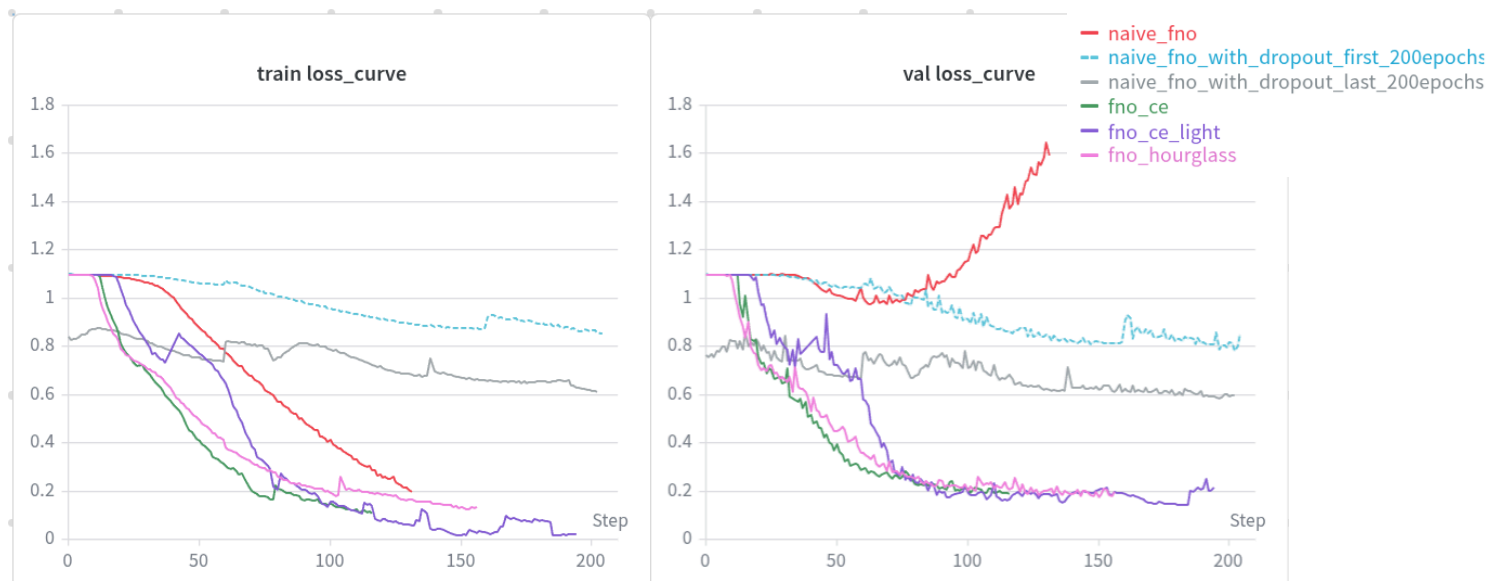
3. **FNO Hourglass** - An encoder-decoder type variant where Fourier modes are progressively reduced in the encoder ($12 \rightarrow 9 \rightarrow 6$) and restored in the decoder ($6 \rightarrow 9 \rightarrow 12$). Skip connections link encoder and decoder stages at corresponding resolution levels. This design creates an information bottleneck in frequency space, analogous to U-Net's spatial bottleneck, achieving competitive performance with approximately half the parameters compared to FNO_ce_light (4M vs 7M)

Hourglass FNO



Note: *Best AUC achieved by FNO_ce_light (0.9954). CNN baseline achieved 1.0*

Architecture	#params	Additional Training Config	Val F1 Score	Val micro avg AUC Score	Classwise AUC
ResNet-18 + SE Blocks	13.6M	NA	-	0.9908	- no: 0.9905 - sphere:0.9824 - vort:0.9936
Naive FNO	7.1M	Spatial dropout = 0.1, cosineannealing scheduler	0.73	0.9019	- no: 0.9666 - sphere:0.8276 - vort:0.8893
FNO	9.7M	Modes = 14	0.93	0.9909	- no: 0.9954 - sphere:0.9808 - vort:0.9912
FNO_light	7.1M	Modes = 12	0.96	0.9954	no: 0.9972 sphere:0.9901 vort:0.9960
FNO_Hourglass	4.3M	NA	0.94	0.9918	- no: 0.9957 - sphere:0.9830 - vort:0.9925



Discussion - I began with naive FNO to establish baseline behavior. With 16 modes and no regularization, the model quickly overfit on the training data. The train - val gap pointed to overcapacity rather than data mismatch, as both distributions were similar.

Reducing modes from 16 to 12 and adding spatial dropout removed overfitting. Initial runs used $\sim 1e-4$ with cosine annealing. FNO performed better at higher rates ($\sim 8e-4$), while cosine restarts caused loss spikes. Hence for following runs I switched to higher lr and StepLR scheduler. However, naive_FNO converged around 0.73 F1 score (less than the CNN baseline). On comparing I noted two things - FNOs have no sense of position and unlike ResNet naive FNO has no skip connection.

Observation

1. FNO requires positional information and skip connections.
2. Higher learning rates work better
3. Hourglass arch is parameter efficient - competitive performance at $\sim 50\%$ parameter count suggests frequency bottlenecks are viable for this task.

4 Project Deliverables

This project focuses on applying neural operators to gravitational lensing simulation. I will begin with a review of existing literature, including ray-tracing-based methods and prior ML4Sci work on diffusion-based simulation pipelines.

Next, I will curate a dataset suitable for operator learning, consisting of source light profiles, mass distributions, and corresponding observed lensed images. The primary focus will be on developing a Fourier Neural Operator (FNO) and benchmarking it against DeepONet, conventional ray-tracing methods, and diffusion-based pipelines.

4.1 Objective and Implementation details

The objective of this project is to develop a Fourier Neural Operator (FNO) model for gravitational lensing simulation, offering an efficient and accurate alternative to traditional ray-tracing methods. The core deliverables include:

4.1.1 Literature Review

- **Review operator learning frameworks:** Focus on DeepONets and standard Neural Operators.
- **Examine FNO variants:** Study the architectural mechanics of Factorized FNO [\[12\]](#) , U-FNO [\[13\]](#) , and DiffFNO [\[14\]](#).
- **Review existing pipelines:** Evaluate traditional simulators like [lenstronomy](#) and [PyAutoLens](#), alongside prior ML4Sci diffusion-based pipelines [\[15\]](#) [\[16\]](#)
- **Investigate PINNs:** Review Physics-Informed Neural Network methods for image-to-image mapping

4.1.2 Dataset Preparation

- **Curate the primary dataset:** Generate standardized pairs of mass distributions, source profiles, and observed lensed images.
- **Build the preprocessing pipeline:** Implement data loading pipelines featuring physics-informed augmentations.
- **Isolate test sets:** Create strict hold-out subsets to evaluate model generalization across different mass profiles and source types.

4.1.3 Core Training Phase

- **Consolidate the codebase:** Refactor and structure the initial FNO code developed during the evaluation tasks.
- **Optimize hyperparameters:** Systematically tune the learning rate, optimizers, schedulers, and Fourier frequency modes.
- **DeepONet code:** Implement and train a vanilla DeepONet to serve as a baseline comparison.
- **Explore physics informed losses:** Experiment with integrating physics-informed loss [subject to experimentation]

4.1.4 Advanced FNO Experimentation

- **Implement variants:** Extend the baseline FNO to evaluate advanced structural variants.
- **Test specific models:** Systematically train and compare Factorized FNO and U-FNO to identify the optimal architecture for lensing.

4.1.5 Evaluation Metrics

- **Measure reconstruction quality:** Quantify image accuracy using SSIM, FID, and LPIPS metrics.
- **Profile computational efficiency:** Track inference latency, peak memory consumption, and GPU batch throughput.
- **Optimize inference speed:** Implement mixed-precision training (FP16) and TorchScript compilation to maximize performance. If time permits, explore GPU level training optimisations as well.

4.1.6 Physics & Generalization Benchmarking

- **Compare architectures:** Benchmark FNO variants and DeepONets.
- **Verify physics preservation:** Ensure the model accurately preserves underlying physical quantities in the generated images.
- **Test parameter generalization:** Evaluate robustness across varying redshifts, source morphologies, Einstein radii, and ellipticities/shear.
- **Validate resolution invariance:** Train the model strictly on 64x64 grids, then test zero-shot inference on 128x128 and 256x256 grids to prove the FNO's theoretical resolution independence.

4.1.7 Documentation

- **Maintain code quality:** Write clear, comprehensive docstrings for all functions, detailing inputs, outputs, and key algorithmic choices.
- **Publish progress:** Document the methodology, architectural decisions, and technical challenges through regular blog-style write-ups.

4.1.8 Extension

If time permits, I would like to explore hybrid operators - probably diffusion + FNO (Diff FNO). Previous projects have explored diffusion, it would be a good extension to combine those learning with a native neural operator or maybe explore diffusion in the spectral domain.

4.2 Detailed Timeline

Community Bonding Period Engage with mentors and ML4SCI community to align on project scope and priorities. Review prior DeepLense work (diffusion-based simulation pipelines, existing lensing datasets). Set up development environment with PyTorch, neuraloperator library, and Lenstronomy/PyAutoLens for ground-truth generation. Finalize dataset specifications with mentors.

Phase	Period	Task
Community Bonding Period		
Literature Review	Week 1	- Read about Neural Operator theory and existing methods. - Review ML4SCI's prior diffusion-based simulation work. - Conduct a focused review of FNO variants, DeepONet implementations. - Review PINN-based image-to-image mapping approaches relevant to inverse problems. - Compile findings into a reference document for later design decisions.
Dataset Preparation	Week 2 Week 3	- Study Lenstronomy and PyAutoLens simulation APIs - understand how mass profiles and source morphologies are parametrized. - Explore existing datasets of gravitational images. This can be used to expand our simulated data. - Draft data generation scripts. - Generate training dataset - Build preprocessing pipeline using physics valid augmentations and normalisation techniques. - Create held-out test sets stratified by mass profile and source morphology for generalization evaluation.
Code preparation and Training	Week 4 Week 5	- Consolidate and refactor the base FNO codebase developed during the evaluation period. - Train the base FNO on Task 1 ($\kappa \rightarrow I_{lensed}$) and Task 2 ($(\kappa, I_{source}) \rightarrow I_{obs}$) - Test with varying modes and hourglass architecture. - Implement and train the vanilla DeepONet to establish the baseline performance. - Conduct an initial performance comparison between the FNO and the DeepONet baseline
Physics Informed Losses	Week 6	- Implement and test custom loss functions both in spatial and spectral domain [gradient penalties, flux conservation, lens equation based loss] - Document the impact of these losses on training stability and output quality.

		Finalize the base FNO model by conducting hyperparameter tuning (learning rate, frequency modes) with custom loss.
Mid Term Evaluation - Prepare the Phase 1 evaluation report for mentors, highlighting the impact of the physics-informed losses. - Publish the first blog post detailing the dataset generation, DeepONet vs. FNO baselines, and initial findings.		
Alternate Architecture	Week 7	- Implement and train FNO variants - Factorised FNO and UFNO - Compare against base FNO.
Evaluation	Week 8 Week 9	- Test generalization on out-of-distribution redshifts and source morphologies. - Perform statistical analysis of geometric properties of generated images against real data and ray-traced simulated data. - Run image classification model trained for specific task 4 on simulated data - Test resolution invariance on 128x128 and 256x256 grids (trained on 64x64).
Benchmarking	Week 10	- Profile simulation speed of all trained models (inference latency, batch throughput) against numerical solvers. - Select the final, optimal FNO architecture based on speed and reconstruction accuracy.
Optimisation	Week 11	- Implement PyTorch-specific optimizations: integrate Automatic Mixed Precision (AMP/FP16) - Utilize <code>torch.compile</code> to capture CUDA operation graphs for faster execution. - Optimize dataloaders for faster GPU throughput. - Export the final model to ONNX for inference benchmarking.
Documentation	Week 12	Complete code-level documentation, ensuring all functions have clear docstrings and the repository includes a reproducible <code>README.md</code>

		• Compile the final GSoC research report, summarizing metrics, optimizations, and the speedup achieved over traditional simulators. • Publish the final blog post and discuss potential post-GSoC project maintenance or paper publication with mentors.
--	--	---

5 Other Information

5.1 Why ML4Sci?

I am applying exclusively to DeepLense because it directly aligns my technical background with my interest in pure science and astronomy. ML4SCI bridges deep learning, research, and open source offering opportunity to work on hard problems. I have practical experience building and optimizing complex architectures - including transformers, diffusion models, and voice AI pipelines - and I want to apply these exact skills to gravitational lensing.

My long-term aspiration is to be involved in applied ML research and contribute to building scalable ML systems for scientific discovery with leading institutions like *CERN*. I also intend to pursue *advanced studies* (probably a PhD) in this direction. Collaborating with organizations like DeepLense places me among like-minded researchers and mentors whose experience and guidance can help me grow - both technically and personally.

5.2 Relevant Experience

I am proficient in **Python** and **PyTorch**, with a strong foundation in deep learning.

- **Computer Vision Research Intern – CANDLE Research Lab, IIT Roorkee:** Worked on evaluating state-of-the-art object detection architectures and designing a lightweight object detection model. This work contributed to our research paper [ConstructNet](#).
- **Data Science Intern – Axis Bank (Business Intelligence Unit):** Helped build the bank's first product recommendation engine. Optimized matrix operations on Spark DataFrames and developed a utility library to streamline matrix operation on internal data.
- **Research Intern (Part-Time) – NeurAI Lab, TU Eindhoven:** Worked on LLM-guided sparsity schemes, with research focus on attention mechanisms and MoEification techniques for LLMs. Work involves writing and testing new attention variants.
- **Open Source Contributions:** Contributed to the [MAYA project](#) under Aya Expedition 2024. Contributed to improving PyTorch documentation as part of the PyTorch Docathon 2023.
- **Free Time Coding:** Spend time to understand the internals of automatic differentiation libraries and CUDA programming.

5.3 Commitments

I am proposing a 175-hour project with a core focus on developing and optimizing the FNO based simulator. While I currently work as an Applied AI Engineer, I have a proven track record of balancing my professional responsibilities with active open-source research. I regularly dedicate time to tangential technical pursuits and collaborate within communities like Hugging Face and Cohere, ensuring I can smoothly manage my schedule to fully execute this GSoC project.

The timeline provided is tentative and entirely flexible; I am ready to adjust the schedule or redefine deliverables based on mentor feedback. I have intentionally built buffer periods into the timeline. I plan to utilize this buffer time to proactively explore alternative network architectures and test supplementary code.

I am based in the Indian Standard Time (IST / UTC+5:30) zone and will be readily available for regular synchronous updates via Zoom, Google Meet, or Discord. In the event of any unforeseen circumstances that might temporarily impact my availability, I will communicate with my mentors well in advance and proactively restructure the timeline to prevent any derailment of the project goals. I intend to continue contributing to DeepLense and ML4SCI well beyond the GSoC period. Always happy to discuss future scope of the project and further enhancements.

6 References

1. Lu, Lu, Pengzhan Jin, and George Em Karniadakis. "Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators." *arXiv preprint arXiv:1910.03193* (2019).
2. https://en.wikipedia.org/wiki/Universal_approximation_theorem
3. Kovachki, Nikola, et al. "Neural operator: Learning maps between function spaces with applications to pdes." *Journal of Machine Learning Research* 24.89 (2023): 1-97.
4. Hendrycks, Dan, and Kevin Gimpel. "Gaussian error linear units (gelus)." *arXiv preprint arXiv:1606.08415* (2016).
5. Li, Zongyi, et al. "Fourier neural operator for parametric partial differential equations." *arXiv preprint arXiv:2010.08895* (2020).
6. https://en.wikipedia.org/wiki/Convolution_theorem
7. https://en.wikipedia.org/wiki/Fast_Fourier_transform
8. Birrer, Simon, and Adam Amara. "lenstronomy: Multi-purpose gravitational lens modelling software package." *Physics of the Dark Universe* 22 (2018): 189-201.
9. Nightingale, James, et al. "PyAutoLens: open-source strong gravitational lensing." *arXiv preprint arXiv:2106.01384* (2021).
10. Hu, Jie, Li Shen, and Gang Sun. "Squeeze-and-excitation networks." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2018.
11. <https://github.com/pytorch/pytorch/blob/v2.11.0/torch/nn/init.py#L578>
12. Tran, Alasdair, et al. "Factorized fourier neural operators." *arXiv preprint arXiv:2111.13802* (2021).
13. Wen, Gege, et al. "U-FNO—An enhanced Fourier neural operator-based deep-learning model for multiphase flow." *Advances in Water Resources* 163 (2022): 104180.
14. Liu, Xiaoyi, and Hao Tang. "Diffno: Diffusion fourier neural operator." *Proceedings of the Computer Vision and Pattern Recognition Conference*. 2025.
15. <https://summerofcode.withgoogle.com/archive/2025/projects/uGmyAV1q>
16. <https://summerofcode.withgoogle.com/archive/2025/projects/T27qiqDH>